



Edit with WPS Office

Project Progress Report On

ANALYSING THE SOFTWARE DESIGN DEFECTS
Submitted To



Amity University Uttar Pradesh
in partial fulfilment of the requirement for the award of the degree of
Masters of Technology
in
Computer Science & Engineering
By

Ekansh
A23009220012
2MTCSE

Batch 2022-2024

Under the guidance of
Dr. Aakanshi Gupta
Department of Computer Science and Engineering,
Amity School Of Engineering and Technology
University Uttar Pradesh,
Noida(U.P)
April 2023



ABSTRACT

Code smells are a group of typical code errors that indicate possible design or implementation problems in software development. They are typically symptoms of deeper problems within the codebase that may lead to reduced maintainability, increased complexity, and higher risk of errors and bugs. A Systematic Literature Review has been performed by accessing the digital libraries that include the publications since 2013 to 2023. We thoroughly studied 23 research papers that were most relevant. The purpose of this paper is to provide a summary of the existing research on code smells, analysing the different machine learning algorithms and their efficiencies. Our findings show that most of the studies have been done only on the code smells detection tools, yet there are inconsistency in the results that these tools provide. Our focus is to find a tool/algorithm which provides consistent results independent of the platforms that it is used in and to enhance and build upon the existing tools to detect code smells.

Code smells are not always bad, and sometimes they may be necessary in order to optimize performance or meet specific requirements. However, they should be carefully considered and addressed as part of a larger effort to raise the overall standard of the codebase.

Keywords: Code Smells, Machine Learning, Java, Decision Trees Algorithm.



CHAPTER 1

INTRODUCTION

Code smells are certain characteristics or symptoms in the source code that suggest the presence of deeper problems. They are called "smells" because they are not necessarily errors, but rather signs that something might be wrong or inefficient. Code smells often indicate violations of good design and coding practices, and can lead to maintenance problems, and performance issues. Code that is poorly implemented is challenging to maintain. Code Smells can be duplicated code, long methods or classes, overly complex or nested conditional statements, large numbers of parameters or dependencies, and inconsistent naming conventions. These code smells are not necessarily bugs, but rather indications of potential problems that may making it more challenging to keep or extend the line of code. Code smells are not always bad, and sometimes they may be necessary in order to optimize performance or meet specific requirements. However, they should be carefully considered and addressed as part of a larger effort to improve the overall quality of the codebase. The goal is to evaluate all machine learning techniques used for detecting code smells.

The largest problem with detecting code smells is that they are poorly defined, leading developers to differ on whether a given piece of code is smelly or not.

Our focus will be to check whether decision trees are a good technique to detect code smells or not. And also to identify other better algorithms that are effective in locating the code smells.



1. Objectives to be accomplished

Our main objective is to study and compare the different algorithms that are currently out there which are useful in detecting code smells. Also, we will be using different tools on same source code to check which existing tool is better in detecting code smells. What kinds of code smells the tools are detecting. What are the limitations of those particular tools. Comparison between the results of the tools used.

2. Different Types of Code Smells

Table 1: Different Code Smells

S.No	ID	Name	Description
1	SM1	Blob	Another name for it is God Class. The term "class" refers to a class that tends to centralise the system's intellect.
2	SM2	Long Parameter List (LPL1)	LPL is an excessively large parameter list that is challenging to comprehend.
3	SM3	Long Method (LM1)	It is a lengthy procedure that is challenging to comprehend, modify, or extend.
4	SM4	Feature Envy (FE1)	It indicates that a method is more focused on classes other than the one in which it is currently situated.
5	SM5	Duplicated Code	It requires changes to be made in multiple places.
6	SM6	Message Chains	Where a chain of method calls is used to access an object's properties or methods, suggesting that the code should be moved to the object itself
7	SM7	Inappropriate dependency	Where two classes have too many dependencies on each other.
8	SM8	Dead Code	Code which is not executed and has no purpose

The above table shows major code smells that are currently identified with their descriptions.

3. Different Tools for Code Smells Detection

Following are the different tools that are most used in detecting code smells.

SonarQube: SonarQube is a popular static analysis tool that can detect code smells, bugs, vulnerabilities, and security issues in a codebase. It uses a set of predefined rules and checks the code against them to identify code smells.

CodeClimate: CodeClimate is another popular static analysis tool that can detect code smells, security issues, and maintainability issues in a codebase. It uses a set of predefined rules and provides suggestions on how to improve the code.

PMD: An open-source programme called PMD may examine Java and other programming languages to find potential flaws, performance problems, and code smells. It checks the code using a set of predetermined rules and offers a thorough report of any faults discovered.



ESLint: ESLint is a popular linting tool for JavaScript that can identify code smells, coding errors, and maintainability issues in a codebase. It provides a set of predefined rules, and users can also write custom rules to suit their specific needs.

ReSharper: ReSharper is a Visual Studio extension that can detect code smells, coding errors, and performance issues in .NET code. It uses a set of predefined rules and provides suggestions on how to fix the issues.

Codacy: Codacy is a cloud-based tool that can detect code smells, security issues, and maintainability issues in a codebase. It uses a set of predefined rules and provides a detailed report of issues found.

These tools can assist programmers in locating code smells inside a codebase and offering recommendations for how to correct them. It's crucial to keep in mind nevertheless that these programmes aren't flawless and can report false positives or overlook specific difficulties. It is always a good idea to manually check the code to make sure it is error-free and maintainable.



Table 2: Code Smells Detection Tools

Tool	Type	Supported Languages	Code Smells detected	Automated Refactoring	Direct Link to code
JDeodrant	Eclipse Plug-in (v. 3.5 e 3.6)	Java	Feature Envy God Class Long Method Type Checking	Yes	Yes
Stench Blossom	Eclipse Plug-in (v. 3.3)	Java	Data Clumps Feature Envy InstanceOf Long Method Large Class Message Chain Switch Statement Typecast	No	Yes
InFusion	Stand-alone application	Java, C++, C	Cyclic Dependencies (CDs) Brain Method (BMs) Data Class (DC1) Feature Envy (FE1) God Class (GC1) Intensive Coupling (IC1) Missing Template (MT1) Method Refused (MR) Parent Request (PR) Significant Duplication (SD) Shotgun Surgery (SS)	No	No
iPlasma	Stand-alone application	Java, C++	Default: Brain Class (BC1) Brain Method (BMs) Data Class (DC1) Dispersed Coupling (DC2) Feature Envy (FE1) God Class (GC1)	No	No



	n		(BMs) Data Class (DC1) Dispersed Coupling (DC2) Feature Envy (FE1) God Class (GC1) Intensive Coupling (IC1) Shotgun Surgery(SS1) Refused Parent Request (RPR1) Tradition Breaker(TB1) By custom rules: Long Method (LM1) Long Parameter List (LPL1) Speculative Generality (SG1)		
PMD	Eclipse plugin or Stand- alone applicatio n	Java	Large Class Long Method Long Parameter List	No	Yes
DECOR(Black Box)	Stand- alone applicatio n	Java	Large Class Lazy Class Long Method Long Parameter List Refused Parent Request Speculative Generality	No	No



CHAPTER 2

LITERATURE REVIEW

2.1. Search Process

Since 2013, relevant conference proceedings and publications were searched manually. The journals are chosen since it is recognised that they provide literature reviews and empirical studies. The chosen periodicals and conferences are displayed in Table 3.



Table 3: Selected Conferences and Journals

Source	Acronym
2019 International Conference on Intelligent Computing and Control Systems (ICCS)	ICCS
IEEE Access	IEEE
2021 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD)	ICICT4SD
2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)	ICSME
TENCON 2019 - 2019 IEEE Region 10 Conference (TENCON)	TENCON
42nd Yearly Computer Software and Applications Conference (COMPSAC), 2018	COMPSAC
2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI)	ICACCI
2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)	SANER
2020 7th International Conference on Dependable Systems and Their Applications (DSA)	DSA
2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)	ICSE

2.2. Sources of Information

Sources Name	Access Link	Total Number of Papers	No. of papers after title based exclusion	No. of papers after abstract based exclusion	After Full text Reading
IEEEExplore	ieeexplore.ieee.org	623	27	21	17
Springer	www.springer.com	256	18	10	3
Wiley Online Library	onlinelibrary.wiley.com	7	3	2	0
Elsevier	www.sciencedirect.com	283	57	11	3



Science Direct					
----------------	--	--	--	--	--

2.2. Inclusion and Exclusion Criteria

We included articles which were published between Jan 1st 2013 to Apr 28th 2023. We considered the articles which were either published in Conferences or in Journals as mentioned in Table 3.

The articles were included after manually searching them using the keywords "Code Smells, Machine Learning, Java, Decision Trees Algorithm".

The outcomes of the discovered articles after the search process are displayed in Table 4.

Table 4: Results of selected articles

ID	Author	Date	Topic Type	Topic Area	Article type	Refs	Include Practitioner Guidelines	Num. of primary studies
A1	A. Kaur, K. Kaur and S. Jain	2016	Advances in Computing, Communications and Informatics	Predicting software change-proneness with code smells and class imbalance learning	ER			
A2	L. Amorim, E. Costa, N. Antunes, B. Fonseca and M. Ribeiro	2015	Software Reliability Engineering	Evaluating the effectiveness of decision trees for detecting code smells	ER			
A3	N. Pritam et al.	2019	IEEE Access	Assessment of Code Smell for Predicting Class Change Proneness Using Machine Learning	ER			
A4	Assessment of Code Smell for Predicting Class Change Proneness Using Machine Learning	2019	Intelligent Computing and Control Systems	Identification of Code Smell Using Machine Learning	ER			
A5	N. Akhter, S. Rahman and	2021	ICICT4SD	An Anti-Pattern Detection Technique Using Machine Learning to Improve Code	ER			



	K. A. Taher			Quality				
A6	A. Imran	2019	Software Maintenance and Evaluation	Design Smell Detection and Analysis for Open Source Java Software	ER			
A7	A. K. Das, S. Yadav and S. Dhal	2019	TENCON	Detecting Code Smells using Deep Learning	ER			
A8	Y. Wang, S. Hu, L. Yin and X. Zhou	2018	IEEE Access	Using Code Evolution Information to Improve the Quality of Labels in Code Smell Datasets	MA			
A9	I. Kaur and A. Kaur	2021	IEEE Access	A Novel Four-Way Approach Designed With Ensemble Feature Selection for Code Smell Detection	SLR			
A10	K. Kaur and P. Kaur	2017	Advances in Computing, Communications and Informatics	Evaluation of sampling techniques in software fault prediction using metrics and code smells	ER			
A11	V. Lenarduzzi, F. Lomio, H. Huttunen and D. Taibi	2020	SANER	Are SonarQube Rules Inducing Bugs?	CS			
A12	H. Chen et al.	2020	DSA	AdaBoost-based Refused Bequest Code Smell Detection with Synthetic Instances	ER			
A13	J. A. Díaz-Pace, A. Tømmassel and D. Godoy	2018	SCAM	[Research Paper] Towards Anticipation of Architectural Smells Using Link Prediction Techniques,	ER			
A14	J. Y. Khan, M. Tawkat Islam Khondaker, G. Uddin and A. Iqbal	2021	SANER	Automatic Detection of Five API Documentation Smells	SLR			
A15	H. Wang, J. Liu, J. Kang, W. Yin, H. Sun and H. Wang,	2020	ISPA/BDCloud/SocialCom/SustainCom	Feature Envy Detection based on Bi-LSTM with Self-Attention Mechanism	ER			



A16	T. Oo, H. Liu and B. Nyirongo	2018	IEEE Access	Dynamic Ranking of Refactoring Menu Items for Integrated Development Environment	ER			
A17	H. Gulabovska and Z. Porkoláb	2019	Scientific Conference on Informatics	Towards More Sophisticated Static Analysis Methods of Python Programs	MA			
A18	Azeem, M. I., Palomba, F., Shi, L., & Wang, Q.	2019	Information and Software Technology	Machine Learning Techniques for Code Smell Detection	SLR and MA			
A19	Almashtari, N., & Lu, L.	2020	ICCCS	Code Smell Detection Tool for Java Script Programs	MA			
A20	Al-Shaaby, A., Aljama'an, H. & Alshayeb, M.	2020	SPRINGER	Bad Smell Detection Using Machine Learning Techniques: A Systematic Literature Review	SLR			
A21	Arcelli Fontana, F., Mäntylä, M.V., Zanoni, M.	2016	SPRINGER	Comparing and experimenting machine learning techniques for code smell detection	ER			
A22	Alkharabsheh, K., Crespo, Y., Mansoor, E. et al.	2019	SPRINGER	Software Design Smell Detection: a systematic mapping study	SLR			
A23	Arcelli Fontana, F., & Zanoni, M.	2017	SCIENCEDIRECT	Code smell severity classification using machine learning techniques. Knowledge-Based Systems	ER			

Where:

ER: Experience Report

SLR: Systematic Literature Review

MA: Meta Analysis

CS: Case Study

2.3. Quality Evaluation of SLRs:

DARE standards created by Centre for Reviews and Dissemination of York University were used to evaluate each article. The below questions forms the basis of criteria:



QA1: The criteria for including/excluding the articles, is it well defined?

QA2: How likely is it that literature review include all the studies?

QA3: Is the efficiency included for the selected articles?

QA4: Were the fundamental data and studies sufficiently described?

Following is how the questions were scored:

QA1: The inclusion criteria are specifically stated in the study, Ans = Y(Yes); P (partially); N (no); the inclusion criteria are not specifically stated and cannot be inferred easily.

QA2: Y, the authors have either referenced and identified all the journals addressing the topic of interest, searched four or more digital libraries, and used additional search strategies; P, the authors have searched three or four digital libraries without using any additional search strategies; N, the authors have searched no more than two digital libraries or a very limited set of journals.

QA3: The research question incorporates quality issues that are addressed by the study; P, the authors have explicitly defined quality criteria and extracted them from each primary study; N, no explicit quality assessment of individual primary studies has been made.

QA4: Y Each study's information is supplied; P Only a summary of primary study information is presented; N Individual primary study results are not specified.

The formula for scoring was $Y = 1$, $P = 0.5$, and $N = 0$.

Table 5: Quality evaluation of Studies

Study	Article Type	QA1	QA2	QA3	QA4	Total Score	Initial Rater Aggrement
A1	ER	P	N	Y	Y	2.5	4
A2	ER	P	Y	P	P	2	4
A3	ER	P	N	Y	Y	2.5	4
A4	ER	P	P	P	Y	2.5	4
A5	ER	N	P	Y	Y	2.5	4
A6	ER	Y	P	Y	Y	3.5	4
A7	ER	N	N	Y	P	1.5	4
A8	MA	P	P	Y	Y	3	4
A9	SLR	Y	Y	P	P	3	4
A10	ER	P	P	Y	Y	3	4
A11	CS	P	P	P	Y	2.5	4
A12	ER	P	P	Y	P	2.5	4
A13	ER	P	Y	Y	P	3	4
A14	SLR	Y	P	P	P	2.5	4
A15	ER	N	P	Y	Y	2.5	4
A16	ER	P	P	Y	P	2.5	4
A17	MA	Y	P	Y	P	3	4
A18	SLR and MA	Y	Y	Y	P	3.5	4
A19	MA	P	Y	P	P	2.5	4
A20	SLR	Y	P	Y	P	3	4
A21	ER	P	P	Y	P	2.5	4



A22	SLR	Y	P	Y	Y	3.5	4
A23	ER	P	P	Y	Y	3	4

Table 6: Average Quality Scores according to year wise publications

Year	2015	2016	2017	2018	2019	2020	2021
No. of Studies	1	2	2	3	7	5	3
Mean Quality Score	2	2.5	3	2.84	2.86	2.6	2.67
Standard Deviation of Scores	0	0	0	.236	.69	.20	.235

The average quality scores yearly wise shows that the number of papers published increased after year 2017 and now become stable. Average quality scores for the papers also increased in first few years and it has also become stable now.

CHAPTER 3

METHODOLOGY

3.1. Introduction

We focused on decision trees algorithm for this study because using decision trees to detect code smells has some benefits, such as the ability to handle large datasets, learning patterns that can be difficult for humans to detect, and the ability to handle different types of features. The choice of features and parameters, as well as the quality of the data used for training and validation, all affect how well the decision tree model performs. To achieve the greatest model performance, it is crucial to properly select the data and the parameters.

We also used effectiveness metrics to measure effectiveness of the algorithm in achieving it's goals. effectiveness metrics are used to evaluate the effectiveness of software development processes, tools, and methodologies in achieving their objectives. Some common objectives that effectiveness metrics can help measure include software quality, development productivity, and project success.

There are several reasons why effectiveness metrics are used in software engineering:



Improve Quality: Effectiveness metrics can help measure the quality of the software being developed. Software development teams can identify areas for improvement and implement remedial measures by measuring software quality.

Optimize Productivity: Effectiveness metrics can help identify bottlenecks and inefficiencies in the development process. Software development teams can take steps to optimise the development process by measuring productivity, identifying areas for productivity improvement, and reporting their findings.

Measure Progress: Effectiveness metrics can help measure the progress of a project or a development process. By measuring progress, software development teams can identify areas where progress is lagging behind, and take corrective actions to get back on track.

Facilitate Design-Making: Effectiveness metrics can offer important insights into how well a development process, a tool, or a methodology is performing. By providing objective data, effectiveness metrics can help facilitate decision-making, and enable software development teams to make informed decisions.

Ensure Project Success: Effectiveness metrics can help ensure the success of a project by measuring progress, quality, and productivity. By using effectiveness metrics, teams working on software development might spot areas that want improvement., and take actions to ensure project success.

In summary, effectiveness metrics are used in software engineering to measure progress, quality, productivity, and to facilitate decision-making. By using effectiveness metrics, software development teams can identify areas for improvement, optimize productivity, and ensure the success of their projects.

3.2. Decision Trees

A particularly useful characteristic of the decision tree classification technique is that it can be quickly examined by a developer or software engineer working on the project that the method is intended to help. This implies that team members can validate, debate, learn from, and even improve the models. When compared to many other machine learning methods that produce "black box" models, this represents a significant advantage.

With the use of supervised training, the decision tree method creates statistical models for use in data classification and prediction. This indicates that each instance in the training set has a known class. The classification model that was created has a tree-like structure with the root node at the top, internal nodes (non-leaves) that represent test attributes, and terminal nodes (leaves) that represent class labels. Following the path from the root to a leaf, the tree can categorise an instance whose class is still unknown after learning the model's parameters.



The C5.0 algorithm is the most sophisticated and effective member of the decision tree algorithm family. The values of software metrics, such as the number of methods, parameters, and lines of code, concerning the Classes of the pre-annotated training dataset are used to train the algorithm. The rules generated in this manner are founded on these metrics.

An effective machine learning approach for spotting code smells is decision trees. We will utilise decision trees to find code smells in the following manner:

Collect data: Collect a dataset of code samples that are labelled as either containing or not containing the code smell you want to detect. The dataset should include various features of the code, such as code metrics, syntax, and structure.

Pre-process the data: The data is pre-processed to prepare it for use in the decision tree. This may involve feature scaling, feature selection, and handling missing data.

Split the data: Create training and testing sets from the dataset. The decision tree will be trained using the training set, and its performance will be assessed using the testing set.

Develop the decision tree: Utilise the training set to train the decision tree. Based on the characteristics of the code, the decision tree will learn to categorise code samples as either having or not having the desired code smell.

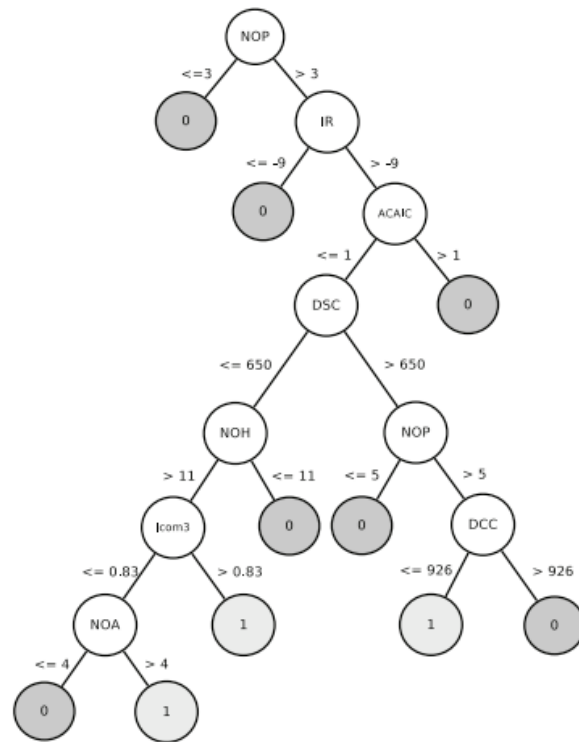
Evaluate the decision tree: Evaluate the performance of the decision tree using the testing set. This will give you an estimate of the decision tree's accuracy in detecting code smells.

Optimize the decision tree: Optimize the decision tree by adjusting its parameters, like the amount of features employed and the depth of the tree. This may enhance the performance of the decision tree.

Use the decision tree: Use the trained decision tree to detect code smells in new code samples. The decision tree will classify the code samples as containing or not containing the code smell based on the features of the code.

It's important to note that there are numerous machine learning algorithms that can be used to identify code smells in addition to decision trees. The dataset's quality, the features employed, and the decision tree's setting settings will all affect how well the decision tree performs. Additionally, depending on the nature of the code smell and the codebase being examined, decision trees may not be appropriate for all sorts of code smells.





3.3. Effectiveness Metrics

We selected Precision, Recall, and F-Measure, three metrics that are frequently used in fields like information retrieval, machine learning, and others that include binary classification.

$$\text{Precision}(P) = TP/(FP+TP)$$

$$\text{Recall}(R) = TP/(FN+TP)$$

Where:

The number of incidents that were accurately classified as positive under the term "True Positives"

The number of negative cases that were mistakenly classified as positive is known as FP: False Positives.

False Negatives (FN) are occurrences of positivity that are mistakenly classified as negatives.

The formula for F-Measure is

F-measure = $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$, and it only represents the harmonic mean between Precision and Recall.

3.4. Data Collection

The gathering of data was crucial to my investigation. The data is the original code, much like with code smells. Additionally, many methods for detecting code smells do not function with all programming languages.

Some techniques for detecting code smells only function with a select few programming languages, whereas other tools operate with a variety of programming languages. So I gathered information on the programming languages Python, Java, and JavaScript.

Below is the raw code for a programme written in Java programming language.

```
public class SmellCheckerS
{
    public static void main(String[] arguments)
    {
        int r=5, i, j, x=1, y=0;
        int[] ar = new int[r];
        int[] arTemp = new int[r];
        ar[0] = 1;
        ar[1] = 1;
        for(i=0; i<r; i++)
        {
            for(j=(r-1); j>i; j--)
                System.out.print(" ");
            for(j=0; j<=i; j++)
            {
                if(i==0)
                    System.out.print("1");
                else
            {
```



```
if(j==0 || j==i)
System.out.print("1 ");
else
{
arTemp[x] = ar[y] + ar[y+1];
System.out.print(arTemp[x]+ " ");
x++;
y++;
}
}
}
System.out.print("\n");
arTemp[x] = 1;
if(i>1)
{
y=0;
ar[y]=1;
for(x=1, y=1; y<=i; x++, y++)
ar[y] = arTemp[x];
x=1;
y=0;
}
}
}
}
```



This above raw code for Java Programming language I used with SonarLint code smells detection tool on platform name Visual Studio.

Above image shows a Java language raw code dataset used with SonarLint smells detection tool on VSCode platform.

3.5. Other Machine Learning Classifiers Used

Table 6 shows the different Machine Learning classifiers that were used to detect code smells.

Algorithm Name	Publication Year	Author
Decision Trees	2019, 2015, 2021	(N. Pritam et al.) and (L. Amorim, E. Costa, N. Antunes, B. Fonseca and M. Ribeiro) and (N. Akhter, S. Rahman and K. A. Taher,)
Genetic Algorithm	2015	L. Amorim, E. Costa, N. Antunes, B. Fonseca and M. Ribeiro
Support Vector Machine	2015, 2021	L. Amorim, E. Costa, N. Antunes, B. Fonseca and M. Ribeiro
Bayesian Beliefs Network	2015,2016	(L. Amorim, E. Costa, N. Antunes, B. Fonseca and M. Ribeiro) and (A. Kaur, K. Kaur and S. Jain)

Naive Bayes	2019,2021	(N. Pritam et al.) and (N. Akhter, S. Rahman and K. A. Taher,)
Random Forest	2019,2021	(N. Pritam et al.) and (N. Akhter, S. Rahman and K. A. Taher,)



CHAPTER 4

RESULTS AND DISCUSSION

4.1. Tools Results

We used code smells detection tools like PMD, Checkstyle, JDeodrant and inFusion to further detect the code smells category wise.

Tools	Analysed Smells				
		Blob	LPL1	LM1	FE1
	inFusion	17	X	X	12
	JDeodrant	43	X	108	49
	Checkstyle	X	1	13	X
	PMD	36	0	14	X

Table: Results for different tools.

The symbol 'X' represents that no code smell were detected, and the numbers represent the amount of code smells detected using different tools.

From these results we came to know that all these tools identify the code smells differently on the same source code.

4.2. Decision Trees Results

As Lucas Amorim and team checked for the results in their article [2], the results are mention below for different types of code smells

Smell Types	Precision (P)	Recall (R)	F-Measure
Blob	0.606	0.424	0.500
Complex Class	0.871	0.842	0.856
Large Class	0.828	0.737	0.779
Long Method (LM1)	0.821	0.827	0.824
Long Parameter List (LPL1)	0.972	0.959	0.965

The findings demonstrate that, for this particular dataset, the Decision Tree approach



employed to identify code smells yields successful results.

However, the recall value for Blob type code smells is 0.424, indicating that the technique is unable to give useful findings in the case of Blob code smells. Recall values below 0.5 indicate that fewer than half of these kinds of code smells were discovered.

CHAPTER 5

FUTURE WORK

In the future, I will be analysing other Machine learning techniques which will be better option to detect code smells. Also, to check whether we can use decision tree algorithm together with other machine learning algorithms like Genetic Algorithm, SVM (Support Vector Machines), etc.

